SOUTH SUDAN DIGITAL DEVELOPMENT HUB

# Project Report

## What is built, how it works, and its licence

| | |
|---|---|
| DOCUMENT | Full project report |
| VERSION | 0.3.0 · demonstration build |
| PREPARED | 8 November 2026 |
| AUTHOR | Mabruk Kei Ring |
| LICENCE | MIT License |
| LIVE BUILD | lastkingmk24.github.io/South-Sudan-Digital-Development-Hub-SSDDH- |

All services, documents and figures in this build are synthetic demonstration data. The site is not an official government service.

CONTENTS

# Contents

Sixteen sections. The first half describes the product a user meets; the second half describes how it is built, deployed and licensed.


| | Section | Page |
|---|---|---|
| 01 | **Executive summary**<br>Purpose, deliverables and an honest account of scope | 3 |
| 02 | **The problem and the approach**<br>Three constraints that shaped every decision | 3 |
| 03 | **What is built — the product surface**<br>The five screens and the record behind each one | 4 |
| 04 | **The service catalogue**<br>Composition, coverage and verification | 5 |
| 05 | **How the directory works**<br>Search, filters and the detail view | 6 |
| 06 | **How community reporting works**<br>Lifecycle, fields and on-device storage | 7 |
| 07 | **How the assistant works**<br>BM25 retrieval, the refusal gate, worked examples | 7 |
| 08 | **Arabic and bilingual design**<br>223 keys, RTL layout and cross-language lookup | 9 |
| 09 | **Offline and installable**<br>Service worker, cache strategy and installation | 10 |
| 10 | **Architecture and file map**<br>Eight modules, no framework, full file map | 10 |
| 11 | **Build, validation and tests**<br>69 tests across four suites, and what they prove | 11 |
| 12 | **Deployment**<br>GitHub Pages, and the constraints it imposes | 12 |
| 13 | **Data, privacy and honesty**<br>What is stored, where it goes, and the honesty rules | 13 |
| 14 | **Limitations and what is not built**<br>Seven gaps, and the full answer on why there is no login | 14 |
| 15 | **Where it goes next**<br>Five stages, in dependency order | 15 |
| 16 | **Licence, copyright and attribution**<br>MIT, copyright, typefaces and reuse | 15 |


> **Reading note.** Numbers in this document were measured against the repository at version 0.3.0, not estimated. Where a figure comes from running the code — the test counts, the assistant's answers, the payload size — the method is named in the text so it can be reproduced.

01

# Executive summary

The South Sudan Digital Development Hub (SSDDH) is a small, complete, offline-first web application that helps people find public services, report problems in their community, and get direct answers from official documents — without a server, an account, or a reliable internet connection.

It is built as a single static site: plain HTML, CSS and JavaScript modules, no framework, no bundler, no database. Everything the product needs travels with the page, so it installs like an app, opens with the network switched off, and runs entirely inside the browser. The whole payload is about 13.0 kB on first load.

| 20 | 10 | 10 | 69 | 0 |
|---|---|---|---|---|
| SERVICES LISTED | STATES COVERED | DOCUMENTS | AUTOMATED TESTS | BACKEND SERVICES |

### What is built

- **A services directory.** Twenty health, education, water, civil-registration, agriculture and legal-aid services across ten states, each with hours, contact details, the services offered and the date the record was last verified.
- **Community reporting.** A four-stage report a resident can file in under a minute — with an optional photo — that is stored on the device and tracked through a visible lifecycle.
- **A citation-gated assistant.** A retrieval system that answers questions by quoting approved documents and attaching the exact passage it used, or refuses outright when nothing in the corpus is good enough.
- **Bilingual interface.** English and Arabic, 223 translated strings in each, with a genuine right-to-left layout rather than a mirrored afterthought.
- **Offline and installable.** A service worker caches the shell and data, so the product works on a phone with no signal and can be added to the home screen.

### What is deliberately not built

There is no backend, no database, no account system and no login. That is not an omission to be fixed later with a quick patch — it is a consequence of the hosting model, and Section 14 explains it in full. Reports and preferences live in the browser on the device that created them. A resident who clears site data loses their reports, and no operator anywhere can see them.

> **All data in this build is demonstration data.** Every service, phone number, document and approval date is synthetic and labelled **[DEMO]** in the interface. Nothing here describes a real facility, and the site is explicitly not an official government service.

02

# The problem and the approach

Public-service information in South Sudan is scattered and hard to act on. A citizen who needs to register a birth may be told three different things by three different offices, and there is no quick way to check which one is current. Information arrives by word of mouth, by radio, or in a printed notice that is already out of date. Meanwhile the connectivity assumptions that most civic-tech products make — an always-on connection, a capable phone, a login — do not hold.

Three constraints shaped every design decision in this project, and they are worth stating plainly because they explain a lot of what follows.

### Constraint 1 — the network is not there

Mobile data is expensive and intermittent, and electricity is unreliable. A product that needs a round-trip to show a phone number is a product that fails in the place it is meant for. So the data ships with the page and the page is cached: after the first visit, the product does not need the network at all.

### Constraint 2 — there is nothing to log in to

Accounts need a server to hold sessions, hashed passwords and personal records. This project is hosted on GitHub Pages, which serves static files and nothing else. A login built in the browser would be a password check written in JavaScript that anyone can read and bypass — security theatre that would make the product look official without protecting anyone. The honest answer is no accounts, and a data model that does not need them.

### Constraint 3 — trust has to be visible

An information product is only useful if a resident can tell how much to trust a given line. That is why every service record carries the date it was last verified and a verified flag, every assistant answer carries the passage and document it came from, and the assistant refuses rather than guessing. Trust is a feature you can see, not a claim in a footer.

### The approach in one line

> Ship the entire product as static files, cache them on the device, and put every decision that matters — search, answering, reporting, translation — into deterministic client-side code that a reader can audit.

03

# What is built — the product surface

The application has five screens in a bottom navigation bar, plus a settings sheet. Every screen is rendered from the same shell, so the interface never reloads and the cached state survives navigation.

| Screen | What it does | Key behaviour |
|---|---|---|
| **Home** | Summary of the catalogue plus the three active alerts and quick links. | Renders instantly from cached JSON; no network call. |
| **Services** | Searchable, filterable directory of all 20 records. | Full-text match on name, city and services; filter by category and state. |
| **Report** | Form to file a community report with category, location, text and optional photo. | Validates 20–1000 characters; photo downscaled to 1024 px before storing. |
| **Assistant** | Ask a question about public services in plain English or Arabic. | Answers with cited passages, or refuses with a visible reason. |
| **More** | Settings, language, data export, privacy notice, about. | Language switch flips the document to RTL live. |

Table 1 — The five screens and what each one guarantees.

### Interaction details that carry weight

- **Search is local and synchronous.** Typing filters an in-memory array; there is no debounce theatre and no request to wait for.
- **Filters compose.** Category and state can be applied together, and the result count updates immediately.
- **The report form remembers nothing it should not.** It validates before saving, downscales photos on the device, and never transmits anything.
- **Language is a first-class setting.** Switching to Arabic sets **dir="rtl"** on the document, mirrors the layout, and translates every visible string from the same 223-key table.
- **Everything degrades.** If IndexedDB is unavailable the photo attachment becomes a silent no-op rather than a crash — there is a test for exactly that.

### The service record

Every entry in the directory has the same shape, which is what makes the list filterable and the detail view predictable. A record holds: name in English and Arabic, category, state, city, street address, phone, opening hours, the list of services offered, the date last verified, a verified flag, a demo source marker, and coordinates for distance sorting.

Here is one record as it is actually stored in the repository — the first entry in **web/assets/js/content.js**:

**Juba Central Primary Health Care Centre**
مركز جوبا المركزي للرعاية الصحية الأولية
Health · Central Equatoria · Juba
Hai Malakal, Block 4, Juba
+211 900 000 101 · Mon–Sat, 08:00–17:00
Outpatient consultations · Maternal care · Immunisation · Malaria testing
Last verified 2026-10-28 · verified

04

# The service catalogue

The directory holds 20 services: 16 are marked verified and 4 are not, which matters more than the count. The verified flag is what the interface uses to tell a resident whether a record has been checked against a source, so the two numbers are shown separately everywhere they appear.

| Category | Records | Share |
|---|---|---|
| **Health** | 5 | 25% |
| **Education** | 5 | 25% |
| **Water** | 3 | 15% |
| **Civil registration** | 3 | 15% |
| **Agriculture** | 2 | 10% |
| **Justice and legal aid** | 2 | 10% |
| **Total** | **20** | **100%** |

Table 2 — Catalogue composition by category.

### Coverage

Ten states are represented, each carrying its Arabic name alongside the English one, so the state filter works in either language. The list is the one the product ships with; it is not a claim that these are the only states in the country.

| State | Arabic name |
|---|---|
| Central Equatoria | الاستوائية الوسطى |
| Western Equatoria | الاستوائية الغربية |
| Eastern Equatoria | الاستوائية الشرقية |
| Jonglei | جونقلي |
| Unity | الوحدة |
| Upper Nile | أعالي النيل |
| Lakes | البحيرات |
| Warrap | واراب |
| Northern Bahr el Ghazal | شمال بحر الغزال |
| Western Bahr el Ghazal | غرب بحر الغزال |

Table 3 — The ten states in the catalogue, with their Arabic names. Arabic is shaped and right-aligned at render time.

### Alerts

Three alert objects sit above the directory. They are ordinary records with a severity, a message in both languages, and a state scope, surfaced on the home screen so a service disruption is visible before the user searches for the service it affects.

05

## How the directory works

The directory is the simplest part of the product and the one most people will use, so it was built to have no failure modes: no request, no loading state, no empty spinner. The data is a JavaScript module imported at load time, and search is an array filter.

### The data path

1. **content.js** exports frozen arrays: **SERVICES**, **STATES**, **DOCUMENTS**, **ALERTS**, **CATEGORIES** and the summary object **STATS**.

2. **app.js** imports those arrays once at module scope and keeps them in memory for the life of the page.
3. A search term and the active filters are reduced to a predicate over each record: it must match the query on name, city or offered services, and satisfy the category and state filters if they are set.
4. The matching subset is rendered. Because the array is small, this happens on every keystroke without perceptible cost.

Normalisation is deliberate rather than incidental: text is lowercased and stripped of diacritics before comparison, so a user typing without accents still finds a record. The same normaliser is shared with the assistant's tokeniser, which is why search and retrieval behave consistently.

### What the detail view shows

Opening a record shows the full address, a tap-to-call phone number, the hours, the service list and — prominently — the verification date. If a record is not verified, the interface says so instead of hiding the field. Distance is computed against the stored coordinates when the browser can supply a location, and the ordering changes accordingly; nothing about the location is sent anywhere.

> **Why no server-side search?** Because a server-side search would require a server. Twenty records and their text fit comfortably in memory, so shipping the index with the page removes a network dependency and a hosting cost at the same time.

06

# How community reporting works

Community reporting is the part of the product that is most clearly a demo and most clearly designed for a real one. A resident describes a problem, optionally attaches a photo, and the report is saved on their device with a reference number and a lifecycle they can follow. What happens on the other side of that lifecycle is, at present, simulated — and Section 14 says so without hedging.

### The four stages

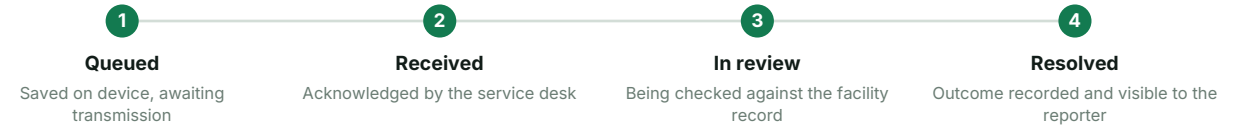



Figure 1 — The report lifecycle. The four stage keys are **queued**, **received**, **in_review** and **resolved**, held in **store.js**.

### What is collected, and what is not

| Field | Rule | Why |
|---|---|---|
| **Category** | One of the six catalogue categories | Routes the report to the right team |
| **Location** | State and city, optional free text | Enough to locate without a GPS trace |
| **Description** | 20 to 1,000 characters, enforced | Long enough to act on, short enough to read |
| **Photo** | Optional, downscaled to 1,024 px, JPEG q0.72 | Keeps storage small on cheap devices |
| **Contact** | Optional, never required | No account, so nothing to verify or leak |
| **Name** | Not collected at all | Nothing to attach the report to |

Table 4 — The report form: six fields, three of them optional, one deliberately absent.

### Storage

Reports and settings are written to **localStorage** as JSON; photos go to IndexedDB as blobs, because localStorage cannot hold binary data at that size. Both are wrapped: if storage is unavailable — private browsing, a full disk, a browser that blocks the API — the calls degrade to no-ops and the rest of the application keeps working. A test asserts this specific behaviour rather than trusting it.

Because the data never leaves the device, the export function matters more than it would in a hosted product: it serialises everything the application stores into a single JSON file with a version marker and an export timestamp, so a resident can carry their reports to a new phone or hand them to someone else.

07

# How the assistant works

The assistant is the most technically interesting part of the build, and the one most likely to be misread. It is **not** a large language model, and it does not call any external service. It is a retrieval engine with a strict refusal rule: it finds the passage in the approved documents that best matches the question, and either quotes it with a citation or declines to answer.

### The pipeline




Figure 2 — The answering path. The gate between retrieval and answering is the feature, not an error path.

### Step 1 — Indexing

The 10 approved documents are split into 37 passages, one per section, each tagged with its document id, document title, authority, approval date and section heading. The index is built once at load time in memory: a term-frequency map per passage plus document frequencies for the whole corpus. There is no inverted index on disk and no pre-computation step.

### Step 2 — Retrieval

Scoring is Okapi BM25 with the standard parameters used in the code: **k1 = 1.5** and **b = 0.75**. The top 4 passages are kept. BM25 is a well-understood ranking function with no training data and no model file, which is exactly the right trade for a corpus this size: it is explainable, it is a few dozen lines, and a reader can check the arithmetic by hand.

### Step 3 — The refusal gate

An answer is only produced when the best passage clears all three thresholds: relative confidence of at least 0.45, a raw BM25 score above 2.2, and either query-term coverage of 0.45 or at least two distinct matched terms. If any of those fails, the assistant returns **no_confident_passage** and the interface shows the refusal as a normal outcome.

This is the design decision that matters most in the whole project. A system that always answers will, on a corpus this small, confidently invent a service that does not exist, a fee that is not charged, or an office that is closed. A system that says “I do not have an approved document that answers this” is less impressive and considerably more useful.

### Step 4 — The answer itself

The answer is extractive: the two highest-scoring sentences from the winning passage are quoted verbatim, with the first letter capitalised. Nothing is paraphrased and nothing is generated, so every word of the answer exists in an approved document. Up to three citations are attached, each carrying the document title, the section heading, the authority, the approval date and a snippet centred on the match.

### Worked example — "What do I need to register a birth?"

> **Answer.** Birth registration is free when the birth is registered within twelve months. A parent or guardian should bring the child health card, the parents national ID copies, and a letter from the village or block leader confirming the birth.

| # | Document | Section | Conf. |
|---|---|---|---|
| 1 | National ID and birth registration | Registering a birth | 1.000 |
| 2 | Business name registration | Who can register | 0.999 |
| 3 | National ID and birth registration | Late registration | 0.895 |

Table 5 — The three citations returned for the question above, as printed by the running code.

Note the second citation: *Business name registration* is not about births, but its "Who can register" section shares vocabulary with the question, so BM25 ranks it highly. The confidence score is relative to the best match, which is why it reads 0.999. This is honest retrieval behaviour rather than a bug, and it is a good argument for showing citations instead of hiding them.

### Worked example — "What is the capital of France?"

> **Answer.** Refused. No approved passage clears the gate, so the interface returns **no_confident_passage** and offers the closest documents rather than guessing. This is the correct behaviour for a corpus that contains no geography reference.

08

# Arabic and bilingual design

Bilingual support is not a translation layer bolted onto an English interface. Arabic changes the geometry of the page: text runs right to left, navigation mirrors, icons that imply direction flip, and numbers stay left-to-right inside the right-to-left flow. The application switches the document direction on the root element and lets CSS handle the rest, using logical properties where possible.

| Layer | English | Arabic |
|---|---|---|
| **Keys per locale** | 223 | 223 |
| **Parity** | identical key sets | verified by build |
| **Direction** | ltr | rtl |
| **Font** | Inter / Source Serif | Noto Naskh Arabic |
| **Numerals** | Western | Western, kept LTR |
| **Layout mirroring** | n/a | CSS logical properties |

Table 6 — The bilingual surface. Key parity is asserted by the build script, not by inspection.

### Cross-language lookup

Arabic attaches its prepositions and conjunctions directly to the noun, so a user who searches for “to register” writes one word, not three. The tokeniser strips the definite article and the common prefixes before matching, which is why a query in Arabic can retrieve an English passage. Table 7 shows four real pairs from the catalogue.

| Surface form | Stem after stripping | Meaning |
|---|---|---|
| لتسجيل | تسجيل | to register |
| المستندات | مستندات | the documents |
| بالمياه | مياه | with the water |
| والمدرسة | مدرسة | and the school |

Table 7 — Arabic prefixes resolved before cross-lingual lookup.

### Why parity is enforced

A missing key in one locale is a hole in the interface that no test-free review will catch, because the English path still works. The build script parses both locales, compares the key sets, and fails the build if they differ. It also checks that every translation key referenced in the JavaScript resolves, which is how the number 223 stopped being a guess and became a fact.

### Arabic in the service data

Each of the twenty services carries a native Arabic name, and each state carries its Arabic name, so a user who switches language reads the same records rather than a transliteration. Table 3 shows all ten pairs as they render in the PDF you are reading — the same shaping the browser performs, done ahead of time because ReportLab has no bidi engine of its own.

09

## Offline and installable

The product is a progressive web app in the strict sense: a manifest, a service worker, an offline fallback, and an install prompt where the browser supports one. It is not “mobile-friendly” — it is installable, and it works on a phone with the radio switched off.

### The cache strategy

| Asset | Strategy | Effect |
|---|---|---|
| **index.html / 404.html** | Precached on install | Shell paints with no network |
| **CSS and JS modules** | Precached on install | No framework CDN, no font CDN |
| **Icons and logo** | Precached on install | Home-screen icon works offline |
| **Service and document data** | Bundled in content.js | Search works offline by construction |
| **User reports** | localStorage / IndexedDB | Survives reload and restart |

Table 8 — What is cached and why. Because the data is a JavaScript module, “offline data” needed no extra mechanism.

The consequence of bundling data as a module is worth spelling out: there is no sync queue, no stale-while-revalidate logic for content, and no possibility of an empty list caused by a failed fetch. The catalogue is present the moment the JavaScript parses, which is before the first paint.

### Install behaviour

- The manifest declares a name, a short name, a start URL, a display mode of **standalone**, a theme colour and 192 px and 512 px icons.
- The install prompt is offered only when the browser fires the relevant event; there is no nagging interstitial.
- When offline, the interface changes one status string and keeps every screen functional — it does not grey out the navigation.
- The service worker is versioned, so a new deployment replaces the cache on the next load rather than leaving users on an old shell indefinitely.

10

# Architecture and file map

There is no framework here, and that is a considered choice rather than a limitation of skill or time. A build step was also avoided: the files in **web/** are the files that are served. What you read in the repository is what runs in the browser.

### Layers

| Layer | Module | Responsibility |
|---|---|---|
| **Shell** | index.html · app.css | Markup and design tokens; no logic |
| **Routing and views** | app.js | Hash router, view rendering, event delegation |
| **Data** | content.js | The catalogue, as frozen arrays |
| **Retrieval** | rag.js | Tokeniser, BM25 index, gate, citations |
| **Persistence** | store.js | Reports, settings, photos, export |
| **Language** | i18n.js | 223 keys in each locale, direction switch |
| **Presentation** | icons.js · util.js | Inline SVG; date, text, distance helpers |
| **Tooling** | scripts/ | Build validation and a local static server |

Table 9 — Eight modules, one dependency direction: presentation depends on data, never the reverse.

### File map

| File | What is in it |
|---|---|
| **index.html** | Application shell; the single page the whole product runs in. |
| **404.html** | Identical shell with a not-found message, for GitHub Pages deep links. |
| **manifest.webmanifest** | Installable-app metadata: name, icons, theme colour, start URL. |
| **sw.js** | Service worker; precaches the shell and serves it offline. |
| **assets/css/app.css** | All styling — design tokens, layout, RTL rules, dark mode. |
| **assets/js/app.js** | Bootstrap, router, view rendering, event wiring. |
| **assets/js/content.js** | The catalogue: services, states, documents, alerts, statistics. |
| **assets/js/rag.js** | Retrieval index, BM25 scoring, extractive answers, refusal gate. |
| **assets/js/store.js** | localStorage and IndexedDB persistence for reports and photos. |
| **assets/js/i18n.js** | English and Arabic strings plus the language switch. |
| **assets/js/icons.js** | Inline SVG icon set, no network fetch. |
| **assets/js/util.js** | Date, text, distance and formatting helpers. |

Table 10 — The 20 files that make up the deployed site (500,662 bytes in total).

### Size

| Measure | Value | Measure | Value |
|---|---|---|---|
| **Files in web/** | 20 | **Bytes in web/** | 500,662 |
| **JavaScript lines** | 3,817 | **CSS lines** | 2,324 |
| **Test lines** | 1,241 | **All source lines** | 7,940 |
| **First-load payload** | 13.0 kB | **Runtime dependencies** | 0 |

Table 11 — Size of the delivered product. The only dependency is a development-time test harness.

11

# Build, validation and tests

There are 69 automated tests across four files, and they run against the real modules in a simulated browser rather than against mocks of them. The suite is the reason several claims in this document can be stated as facts instead of hopes.

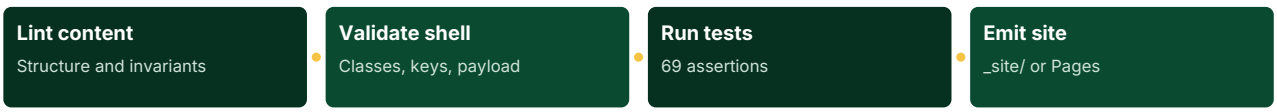



Figure 3 — What **npm run verify** does, in order. Validation and tests share the same modules the browser loads.

| Test file | Area | Tests |
|---|---|---|
| **tests/content.test.mjs** | Content integrity | 21 |
| **tests/shell.test.mjs** | Build and static-shell | 20 |
| **tests/store.test.mjs** | Local persistence | 16 |
| **tests/app.test.mjs** | Interface behaviour | 12 |
| **Total** | | **69** |

Table 12 — The four suites and their counts, as reported by the test runner.

### What the tests actually check

- **Content integrity.** Every service has the required fields, every category id in use exists, every state reference resolves, no duplicate ids, and the recorded statistics match the arrays they summarise.
- **Build and shell.** Every CSS class used in the markup exists in the stylesheet, every translation key referenced in code resolves, English and Arabic key sets are identical, and the payload stays within budget.
- **Local persistence.** Reports round-trip through storage, the lifecycle advances in order, export produces valid JSON, and storage absence is a no-op rather than a throw.
- **Interface behaviour.** Navigation renders the right view, search filters correctly, the language switch flips direction, and the assistant both answers and refuses in the expected cases.

The assistant tests are the interesting ones: one asserts the birth-registration answer and its citation, the other asserts that an off-corpus question returns a refusal. Both a positive and a negative case are pinned, so a future change that makes the assistant answer everything will fail the suite.

12

# Deployment

The site is deployed to GitHub Pages from the repository root, with the build publishing **web/** to the Pages root. A workflow runs the validation and test steps on every push, so a deployment that would break the shell does not reach the live URL.

| Item | Value |
|---|---|
| **Host** | GitHub Pages (static) |
| **Source** | Repository root, publishing web/ |
| **Build command** | node scripts/build.mjs --out _site |
| **Verify command** | node scripts/build.mjs && node --test tests/*.test.mjs |
| **Live URL** | https://lastkingmk24.github.io/South-Sudan-Digital-Development-Hub-SSDDH-/ |
| **Repository** | https://github.com/lastkingmk24/South-Sudan-Digital-Development-Hub-SSDDH- |
| **HTTPS** | Provided by Pages |
| **Server-side code** | None |

Table 13 — Deployment facts. The last row is the one that constrains the whole design.

### Why static hosting is the right constraint

Pages gives free, fast, globally distributed hosting with HTTPS and no operations burden. What it does not give is a process to run, so any feature that needs one — accounts, server-side search, shared reports, an editor for the catalogue — is out of reach until the hosting model changes. Rather than pretend otherwise, the product was designed so that the features which matter most do not need a server at all.

### The honest consequence

A report filed on one device is invisible on another. There is no shared backlog, no administrator view and no way for an agency to read what residents submit. Every report is a note to self with a well-designed lifecycle attached. This is the single largest gap between the current build and a useful public service, and closing it requires a backend — which Section 15 sets out.

13

# Data, privacy and honesty

Because nothing is transmitted, the privacy story is unusually simple: the application cannot leak what it never sends. That is a genuine strength of the current architecture, and it should not be oversold — it is a consequence of having no backend, not a guarantee that would survive one being added.

### What is stored, and where

| Data | Where | Leaves the device? |
|---|---|---|
| **Catalogue** | content.js, cached by the service worker | No |
| **Reports** | localStorage on the device | No |
| **Report photos** | IndexedDB on the device | No |
| **Language preference** | localStorage | No |
| **Location, if granted** | Held in memory for distance sorting | No |
| **Cookies** | None set | n/a |
| **Analytics or trackers** | None included | n/a |

Table 14 — The complete data inventory. There is no third row to hide.

### The honesty rules the interface follows

1. Every synthetic record is marked **[DEMO]**. There is no path through the interface that presents demonstration data as real.
2. The site states that it is not an official government service, in the footer and in the about view.
3. Unverified records say they are unverified rather than omitting the field.
4. The assistant shows its citations and shows its refusals. It has no mode in which it answers without a source.
5. The project documentation describes only what is in the repository. Earlier drafts of the README implied a backend and a mobile client; those were corrected because they had never existed.

> **A note on the word “demo”.** It appears on twenty service records, ten documents and every screenshot in this report. It is not modesty and it is not a disclaimer to skip past: the difference between a demonstration and a public information service is whether a person can act on it. Today, they cannot, and the product says so before they try.

14

# Limitations and what is not built

This section exists because a project document that only lists achievements is a brochure, not a report. The following are real constraints of the current build, stated in the order they are likely to matter to a reader.

| Gap | Consequence | What closing it needs |
|---|---|---|
| **No backend** | Reports cannot reach anyone; services cannot be updated without a redeploy | A server, a database and an operational owner |
| **No accounts or login** | No identity, so no per-user history and no staff access | A server-side session model, which static hosting cannot provide |
| **No shared report queue** | A report exists only on the device that filed it | Server storage plus an intake process |
| **Synthetic catalogue** | Nothing in the directory can be acted on | Verified data from the responsible ministries |
| **Extractive answers only** | The assistant cannot summarise across documents | Either a larger curated corpus or a hosted model with strict grounding |
| **No offline write-back** | A report filed offline stays offline until exported by hand | A sync queue and a server to sync to |
| **No audit trail** | Nothing records who changed what | Server-side versioning of the catalogue |

Table 15 — The seven gaps that stand between this build and a public service.

### Why no login — the full answer

This is the question the project has been asked most often, so it deserves a direct answer rather than a design rationale. GitHub Pages serves files. It does not run code on the server, so there is nowhere to keep a session, nowhere to keep a password hash, and nowhere to check a credential that the client cannot also forge.

A login implemented entirely in the browser would consist of a JavaScript function comparing a typed string against a value shipped in the same bundle. Anyone could read the bundle, skip the function, or edit it. It would not protect a single byte. What it *would* do is teach users that this site has accounts, which is a false claim about a product that has none.

So the honest design is an application with no accounts, storing what it needs on the device, that says clearly that it is a demonstration. When a backend exists, accounts become meaningful: a server can hold the session, hash the password, and decide what a request is allowed to do. Until then, adding a login screen would be a costume, not a feature.

15

# Where it goes next

The order below is deliberate: it follows the dependency chain, not the appeal of the features. Nothing later on the list is worth building before the thing above it exists.

### 1 · Verified content

Replace the synthetic catalogue with data agreed with the responsible ministries, including a named owner and a review date for each record. This is the only step that makes the product useful on its own.

### 2 · A small backend

An intake endpoint for reports, a store for them, and a minimal staff view. A single service and a single table would be enough to turn a note-to-self into a request someone can answer.

### 3 · Identity, done properly

Once there is a server, staff accounts become meaningful: hashed credentials, server-side sessions, and role checks that a client cannot bypass. Resident accounts can stay optional or absent.

### 4 · Editorial tooling

A way for a non-developer to correct a phone number without a pull request. Until this exists, every content fix is a code change.

### 5 · Better answers

A larger curated corpus first; only then consider a hosted model, and only with grounding and citation requirements as strict as the ones already enforced here.

> The current build is, in effect, stage zero: a complete, tested, honest demonstration of the interface and the retrieval model, waiting on content and a backend to become a service.

16

# Licence, copyright and attribution

The project is released under the MIT License. The copyright line, reproduced verbatim from the repository's **LICENSE** file, is:

> **Copyright (c) 2026 Mabruk Kei Ring (Founder, South Sudan Digital Development Hub)**
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

MIT is a permissive licence: anyone may use, modify and redistribute the code, including commercially, provided the copyright and permission notices travel with it. There is no warranty. MIT does not grant rights over the *data* in the demonstration catalogue, which is synthetic and carries no independent value.

### Attribution of the typefaces

The report embeds three typefaces, all under the SIL Open Font License: **Inter** by Rasmus Andersson, **Source Serif 4** by Frank Grießhammer for Adobe, and **Noto Naskh Arabic** by Google. The web application loads no web fonts at all — it uses the system stack, which helps keep its first-load payload near 13 kB.

### How to reuse this work

The repository is at **https://github.com/lastkingmk24/South-Sudan-Digital-Development-Hub-SSDDH-**. Clone it, run **npm install** for the test harness and **npm run verify** to reproduce the results in this document. The live build is at **https://lastkingmk24.github.io/South-Sudan-Digital-Development-Hub-SSDDH-/**.